

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

ЗВІТ
з лабораторної роботи №1

Тема: Технології віддаленого виклику процедур – gRPC

Варіант 17

Київ – 2025

1. Тема роботи

Технології віддаленого виклику процедур – gRPC. Розробка клієнт-серверної програми з використанням gRPC server-side streaming.

2. Мета роботи

Ознайомитися з фреймворком gRPC та форматом серіалізації Protocol Buffers. Реалізувати сервер, який у визначений момент часу розсилає повідомлення підключеним клієнтам через серверний потік (server-side streaming RPC).

3. Індивідуальне завдання

Варіант 17. Сервер розсилає повідомлення у певний час визначеним клієнтам.

4. Теоретичні відомості

gRPC — відкритий фреймворк від Google для організації міжсервісної взаємодії. Побудований поверх HTTP/2, що надає мультиплексування потоків, стиснення заголовків і двонаправлений зв'язок. Контракт між клієнтом і сервером описується у файлах .proto мовою Protocol Buffers; з них автоматично генерується код для обох сторін.

gRPC підтримує чотири типи RPC: unary (один запит — одна відповідь), server-side streaming (один запит — потік відповідей), client-side streaming та bidirectional streaming. У цій роботі використовується server-side streaming: клієнт підписується одним викликом і отримує повідомлення до закриття потоку.

5. Хід виконання роботи

5.1. Proto-контракт

Файл notification.proto визначає сервіс NotificationService з методом SubscribeMessages, що приймає порожній запит (google.protobuf.Empty) і повертає потік повідомлень типу ScheduledMessage.

```
service NotificationService {  
    rpc SubscribeMessages(google.protobuf.Empty)  
        returns (stream ScheduledMessage);  
}  
  
message ScheduledMessage {
```

```
        string content    = 1;
        string timestamp = 2;
    }
```

5.2. Серверна реалізація

Клас `NotificationServiceImpl` розширює згенерований `NotificationServiceGrpc.NotificationServiceImplBase` і перевизначає метод `subscribeMessages`. Сервер у циклі формує об'єкт `ScheduledMessage` з поточним часом і текстом повідомлення, передає його в `StreamObserver` через `onNext()` і робить паузу між відправками. Цикл завершується викликом `onCompleted()` при закритті з'єднання.

5.3. Конфігурація Spring Boot

Застосунок побудовано на Spring Boot 4.0 з модулем `spring-grpc-spring-boot-starter`. gRPC-сервер стартує автоматично разом із контекстом Spring і слухає порт 9090 за замовчуванням. Залежності управляються через Gradle (Kotlin DSL, Java 21).

5.4. Перевірка роботи

Підключення клієнта виконувалось через `grpcurl`. Після виклику методу `SubscribeMessages` у консолі фіксувався безперервний потік об'єктів `ScheduledMessage` з полями `content` і `timestamp`. Потік залишається відкритим до від'єднання клієнта.

6. Висновки

У ході виконання лабораторної роботи реалізовано клієнт-серверний застосунок на базі gRPC із використанням серверного потокового передавання. Засвоєно синтаксис мови Protocol Buffers, принцип кодогенерації з `.proto`-файлів та інтеграцію gRPC з фреймворком Spring Boot. Server-side streaming виявився зручним механізмом для сценаріїв push-сповіщень, де сервер ініціює передачу даних без повторних запитів з боку клієнта.

7. Посилання

Репозиторій з вихідним кодом:
<https://github.com/javaAndScriptDeveloper/lolitech/tree/main/defi/submission1>